Ansible và tự động hóa máy chủ

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

Ansible và tự động hóa cấu hình máy

Terraform thường quản lý vòng đời tài nguyên như network, VM hoặc dịch vụ cloud. Ansible thường dùng inventory và playbook để cấu hình hệ điều hành/phần mềm trên các máy. Một playbook mô tả trạng thái mong muốn bằng module; chạy lại nên đưa hệ thống về cùng kết quả nếu không có thay đổi đầu vào. Đây là tính idempotent cần kiểm tra, không phải chỉ chạy thành công một lần. Ansible cần kết nối và quyền trên managed node; kiểm soát inventory để tránh chạy vào sai môi trường.

Lab Ansible an toàn

Khi có một VM Linux lab, tạo inventory chỉ chứa VM đó, viết playbook kiểm tra kết nối và tạo một file dấu trong thư mục lab theo tài liệu Ansible. Chạy lần một, ghi trạng thái changed; chạy lần hai, mong đợi không còn thay đổi nếu module dùng đúng. Sửa nội dung mong muốn, chạy lại và quan sát diff. Không dùng playbook cài package hoặc restart service trên máy chung trước khi đọc task và target inventory.

Ansible, kubespray và ranh giới với Terraform

Terraform quản lý resource qua provider và state: network, VM, cluster managed, tài nguyên cloud. Ansible chạy playbook idempotent để cấu hình hệ điều hành, cài gói, tạo file, thiết lập service. Trong on-prem, có thể dùng Terraform tạo VM, Ansible cấu hình VM rồi kubespray (tập playbook Ansible) dựng Kubernetes. Trình tự này không tuyệt đối; sản phẩm cloud managed sẽ khác. Điều cốt lõi là phân định ownership để Terraform và Ansible không cùng sửa một cấu hình theo hai hướng.

Idempotent nghĩa chạy lại với cùng desired state không tạo thay đổi ngoài ý muốn. Kiểm chứng bằng cách chạy playbook lần hai và xem số task **changed**. Không coi **changed=0** là chứng minh ứng dụng khỏe; vẫn cần health check. Quản lý inventory, secret (Ansible Vault hoặc secret manager), version collection/role và giới hạn host để tránh chạy nhầm môi trường. Với kubespray, phải xem ma trận phiên bản Kubernetes, OS, container runtime và network plugin trước khi triển khai.

Tình huống phỏng vấn về release microservices

Dịch vụ A gọi B. B đổi API không tương thích. Nếu deploy A trước B hoặc rollback B sau khi A đã đổi, request lỗi dù từng pipeline đều xanh. Hãy nêu contract test, triển khai backward compatible, canary hoặc thứ tự release, theo dõi metric theo phiên bản và cách dừng. Một release an toàn là thuộc tính của **toàn hệ thống**, không chỉ của từng image.

Mô hình điều khiển và inventory

Ansible chạy từ control node tới managed node thường qua SSH/WinRM, dùng inventory xác định host/group và playbook mô tả task. Bạn không cần agent chạy nền trên từng Linux host trong mô hình phổ biến. Một playbook đúng cần xác định host nào, user/quyền nào, biến nào, và task nào được phép thay đổi. **hosts: all** với inventory production rộng là rủi ro nếu chưa xem phạm vi; dùng group riêng và **--limit** cho lab. Ansible Vault mã hóa dữ liệu nhạy cảm trong file nhưng không tự giải quyết mọi vấn đề về phân quyền, rotation hay secret trong log.

Idempotent là chạy lại với cùng desired state mà không tạo thay đổi ngoài ý muốn. Dùng module như **file**, **template**, **package**, **service** thay vì shell command không có điều kiện nếu có thể. Handler chỉ chạy khi task notify có thay đổi, ví dụ template Nginx thay thì reload service. Cần kiểm tra template trước reload để tránh đưa config lỗi vào production. **check mode**/**diff** giúp preview nhưng không bảo đảm mọi module mô phỏng chính xác; review và lab vẫn cần thiết.

Lab an toàn trên một VM

Tạo inventory chỉ có VM lab và xác nhận IP/SSH user. Viết playbook tạo **/tmp/devops-mentor-lab** và một file text với nội dung do biến **lesson_version** điều khiển. Chạy **ansible-playbook -i inventory.ini playbook.yml --limit lab**; ghi số task **ok**/**changed**. Chạy lần hai, kỳ vọng **changed=0**. Đổi **lesson_version**, dùng **--check --diff** để xem dự đoán rồi chạy thật. Kiểm tra nội dung file trên VM. Cuối lab, dùng playbook riêng xóa đúng file lab; không chạy task xóa cả thư mục nếu chưa xác nhận đường dẫn.

```
- hosts: lab
  become: false
  vars:
    lesson_version: "v1"
  tasks:
    - name: Create lab directory
      ansible.builtin.file:
        path: /tmp/devops-mentor-lab
        state: directory
        mode: '0755'
    - name: Write version marker
      ansible.builtin.copy:
        dest: /tmp/devops-mentor-lab/version.txt
        content: "{{ lesson_version }}\n"
        mode: '0644'
```

Nếu task thứ hai luôn báo changed, đọc module/biến và nội dung file trước khi chấp nhận. Một playbook **changed=0** chưa chứng minh service khỏe; chạy health check ngoài playbook hoặc module kiểm tra phù hợp. Với nhiều host, dùng rolling batches/serial và health check giữa các nhóm để tránh restart toàn bộ fleet cùng lúc.

Ansible trong hạ tầng hybrid

Trên VMware/cloud, VM được tạo bằng một công cụ IaC, sau đó Ansible cấu hình OS, user, Nginx/HAProxy hoặc agent monitoring. Inventory có thể tĩnh cho lab, động cho fleet thay đổi; cần tránh host cũ/IP tái sử dụng dẫn tới chạy sai máy. Với kubespray, các playbook cài Kubernetes trên nhiều node, nhưng bạn vẫn phải quản lý phiên bản OS/runtime, backup control plane, etcd, storage/network plugin và upgrade. Không coi "playbook xanh" là bằng chứng cluster production HA.

Case: playbook update Nginx template trên 20 host, reload đồng thời, 30% request lỗi. Phân tích vì sao cần validate config trước reload, chạy từng batch, tách LB backend đang drain, kiểm tra health sau mỗi batch và có bản config trước để rollback. Ranh giới kiểm soát: Ansible thay file/service; LB và ứng dụng vẫn cần telemetry để xác nhận người dùng không bị ảnh hưởng.

Ansible là vòng lặp cấu hình qua inventory

Inventory định nghĩa host/group và biến; playbook mô tả task mong muốn; controller kết nối host qua SSH/WinRM tùy loại và module thực hiện thay đổi. Một task **package state=present** có thể kiểm tra gói đã cài rồi không đổi gì; một task **shell: apt install ...** tùy tiện có thể khó idempotent và khó hiểu changed status. Playbook tốt cho phép chạy lại để sửa drift mà không tạo tác động bất ngờ. Nhưng idempotency là thuộc tính của task/module/thực tế, không tự động có chỉ vì file viết bằng YAML.

Thứ tự biến trong Ansible có nhiều tầng và có thể phức tạp; khi một giá trị sai, ghi nguồn của biến (inventory, group_vars, host_vars, role defaults, extra vars...) thay vì sửa bừa ở nơi đầu tiên thấy. Template Jinja2 có thể tạo config khác nhau theo host; render output và so diff trước khi restart service. Handler chỉ chạy khi task notify thay đổi theo semantics của module; nếu task luôn **changed**, service có thể restart mỗi run. Với hệ thống production, **serial** giới hạn số host thay đồng thời; health check trước/sau từng batch và đường rollback quan trọng hơn chạy nhanh.

Lab local an toàn: tạo file cấu hình có kiểm chứng

Điều kiện: Ansible trên Linux/WSL hoặc VM lab; dùng inventory chỉ trỏ **localhost ansible_connection=local**, không trỏ server công ty. Tạo playbook gồm task **file** tạo thư mục lab trong home, **copy** ghi file **app.conf** có **PORT=8765**, và **debug** in đường file (không in secret). Chạy **ansible-playbook -i inventory.ini playbook.yml --check --diff**, rồi chạy thật. **Kết quả dự kiến:** check mode báo dự kiến changed nếu module hỗ trợ, run thật tạo file. Chạy lần hai: các task idempotent nên **changed=0** trong điều kiện ổn định. Nếu không, điều tra task nào luôn đổi và vì sao.

Sửa port thành 8766, chạy **--check --diff** để thấy trước/sau, rồi apply và kiểm tra nội dung. Nếu file config là secret, **không dùng diff/log công khai**; dùng Vault/secret manager và **no_log** theo phạm vi cần thiết. Dọn đúng thư mục lab của bạn sau khi xác nhận. Không thử playbook với **hosts: all** khi inventory mặc định có máy khác; luôn in **--list-hosts** trước một run có tác động.

Giới hạn check mode

Ansible docs nói check mode là mô phỏng; module không hỗ trợ có thể bỏ qua hoặc không báo đầy đủ. Task phụ thuộc output của task trước có thể cho kết quả khác run thật. Vì thế **--check --diff** là guardrail, không là bằng chứng duy nhất. Kiểm chứng sau apply bằng state/service/HTTP request thật. Một config file hợp lệ về text vẫn có thể làm Nginx không reload được; thêm validate command phù hợp trước khi thay file và handler reload chỉ khi cần.

Ansible với Terraform và kubespray

Terraform phù hợp quản lý tài nguyên cloud/VM và state; Ansible phù hợp cấu hình OS/service trên host; kubespray dùng Ansible để triển khai cluster Kubernetes theo inventory/role và phiên bản dự án. Nếu Terraform và Ansible cùng quản lý một thuộc tính (ví dụ cloud-init user và Ansible user file), xác định owner để tránh ghi đè. Với kubespray, thay inventory/biến có thể tác động control plane/CNI/etcd; không chạy playbook upgrade trên production chỉ vì lab lên cluster thành công. Đọc version matrix, backup etcd và thử staging.

Case có lời giải: run playbook xong web server mất traffic

Play recap có **changed=8, failed=0**, nhưng Nginx trả 502. Đừng kết luận playbook thành công về dịch vụ. So config đã render, **nginx -t**, trạng thái/restart time, upstream address/port, error log và request smoke. Nếu template đổi upstream từ 8000 thành 8001 mà app vẫn nghe 8000, Ansible đã thực thi đúng điều **bạn khai báo**, nhưng khai báo sai. Khôi phục config đã biết tốt theo runbook, xác nhận traffic, rồi sửa biến/test. Một task validate có thể bắt syntax, nhưng không bắt upstream không tồn tại nếu cấu hình vẫn hợp lệ.

Câu hỏi: tại sao **changed=0** ở run thứ hai quan trọng? Nó là một bằng chứng idempotency của điều kiện lab; không chứng minh mọi môi trường. **Câu hỏi: --check** có đủ an toàn tuyệt đối? Không; hành vi phụ thuộc module/playbook và kết quả có thể thiếu. **Câu hỏi:** vì sao dùng **serial**? Để giới hạn blast radius, giữ capacity và quan sát giữa các batch. Nguồn: Ansible playbooks, check/diff mode.

Từ playbook đến trạng thái: một ví dụ đầy đủ

Giả sử cần cấu hình Nginx trên 20 VM. Inventory chia **web_staging** và **web_production**; role cài package phiên bản được phê duyệt, render upstream config, validate bằng **nginx -t**, notify handler reload và gọi smoke test qua LB. Mỗi bước có owner và điều kiện. Nếu cài package “latest” mỗi lần run, bạn có thể thay version ngoài kế hoạch. Nếu handler reload trước khi validate, lỗi syntax gây outage. Nếu smoke test chỉ gọi localhost của một VM, LB tới host khác vẫn có thể lỗi. Playbook cần làm **đúng mục tiêu vận hành**, không chỉ chạy thành công.

Ví dụ pseudocode YAML cho một task an toàn:

```
  - name: Render Nginx upstream config
    ansible.builtin.template:
      src: notes.conf.j2
      dest: /etc/nginx/conf.d/notes.conf
      owner: root
      group: root
      mode: '0644'
    notify: Reload Nginx
```

Đây là **mẫu để đọc**, không phải playbook hoàn chỉnh: nó chưa kiểm tra toàn cấu hình trước khi thay file. Với Nginx, một fragment trong **conf.d** thường phải được kiểm tra trong ngữ cảnh main config, nên không thể tùy tiện dùng **nginx -t -c %s** như thể **%s** là main config. Trong thực tế hãy dùng một bước validation phù hợp với layout config của bạn trên VM lab/staging, rồi chỉ reload khi cấu hình hợp lệ; đọc module docs về **validate** và thử đường rollback. Không copy nguyên đoạn vào production.

Biến, vault và quyền

Một role dùng **db_password** từ vault/secret manager; **debug: var=db_password** hoặc diff mode có thể lộ giá trị. **no_log: true** giảm hiện trong output cho task phù hợp, nhưng không thay quyền truy cập vault và log của ứng dụng. Inventory chứa SSH user/host, có thể nhạy cảm; repo public không nên chứa private key. Dùng **become** chỉ cho task cần root, không chạy toàn bộ playbook đặc quyền nếu không cần. Nếu một playbook trỏ **hosts: all** và inventory production có 500 host, rủi ro blast radius lớn; **--list-hosts**, **--limit**, **serial** và approval là guardrail.

Idempotency và handler qua lỗi có lời giải

Task **shell: date > /etc/notes/reload-marker** luôn đổi file mỗi lần chạy. Nó báo changed mỗi lần; handler restart app mỗi lần, gây gián đoạn. Thay bằng module phù hợp nếu có, hoặc dùng điều kiện **creates**/**changed_when** dựa trên trạng thái thật, nhưng đừng làm **changed_when: false** để che thay đổi thực. Chạy playbook hai lần trong lab, so recap và trạng thái service; lần hai **changed=0** là bằng chứng mong đợi cho cấu hình ổn định. Nếu template chứa timestamp render mỗi lần, file luôn đổi dù cấu hình chức năng không đổi; bỏ timestamp khỏi config hoặc quản lý riêng.

Lỗi network khác lỗi module

UNREACHABLE thường là SSH/DNS/port/auth/host key, chưa tới task cấu hình. **FAILED** ở package có thể là repo/permission/version; **FAILED** ở template có thể biến thiếu; **FAILED** ở handler reload có thể config không hợp lệ. Đọc recap theo host và task đầu tiên fail, không chỉ dòng lỗi cuối. Nếu 2/20 host fail sau batch đầu, dừng rollout, so điểm khác giữa hai host với 18 host khỏe: distro, network, disk, biến inventory, package version. Không retry toàn bộ production mù quáng.

Kế hoạch rollout từng batch

Với 20 web VM, chọn **serial: 2** để thay 2 host một lúc, kiểm tra health từ LB trước khi tiếp. Nhưng nếu LB có minimum healthy backends hoặc traffic không phân đều, 2 có thể vẫn quá nhiều; tính capacity N-1 và quan sát. Nếu host không healthy sau thay đổi, rollback config/package trên batch đó trước khi mở batch tiếp. Cần biết handler/reload timing, connection draining và session state. Ansible có thể phối hợp với LB API nhưng cần quyền/transaction plan. Chạy **--check** trên staging là bước tốt, không thay diễn tập với traffic.

Bài phỏng vấn có đáp án

Ansible khác Terraform ở đâu? Ansible thường tập trung cấu hình host/task qua inventory/module; Terraform theo dõi resource hạ tầng qua state và plan. Có giao nhau; xác định owner để tránh drift. **Kubespray là gì trong ngữ cảnh này?** Bộ playbook/roles triển khai Kubernetes trên hạ tầng, nhưng vận hành cluster production vẫn cần etcd backup, upgrade, CNI/CSI và SLO. **Vì sao playbook ok=20 failed=0 chưa đủ?** Nó chứng minh task thực thi theo module; app có thể trả 502 do cấu hình logic sai, nên smoke test/metrics sau thay đổi là bắt buộc. Nguồn: Ansible desired state/idempotency.

Bài thiết kế role nhiều môi trường

Bạn có staging và production dùng cùng Nginx role. Role default đặt port 8000; group_vars staging đổi 8080, production 8000. Nếu production cũng nhận biến từ host_vars cũ là 9000, kết quả có thể không như bạn nghĩ. Trước run, in inventory/host variables đã resolve cho **một host lab** và render template; ghi nguồn của giá trị cuối. Sau đó thay biến ở đúng lớp owner, không thêm một extra var vào pipeline để che xung đột. Nếu 20 host có config khác nhau không chủ đích, việc quản lý biến đã trở thành vấn đề vận hành.

Tình huống: một host trả **UNREACHABLE**, 19 host chạy xong; nếu pipeline báo xanh vì **ignore_unreachable**, môi trường ở trạng thái nửa mới nửa cũ. Quyết định có chấp nhận degraded

capacity không, có cần rollback 19 host hay sửa SSH cho host thứ 20, và alert nào theo dõi phiên bản từng host. Ghi rõ rollout policy trước khi chạy. Đây là lý do cần batch, health check và inventory ownership chứ không chỉ một playbook idempotent.

Tự kiểm tra: vẽ đường controller -> SSH -> module -> file config -> handler -> Nginx -> LB -> người dùng. Ở mỗi mũi tên ghi một lỗi và lệnh/bằng chứng read-only đầu tiên. Nếu chỉ kiểm tra recap Ansible mà không kiểm tra LB request, bạn mới chứng minh lớp cấu hình, chưa chứng minh dịch vụ.

Ranh giới với Windows và PowerShell

Trong JD Azure/hybrid, Ansible có thể quản lý host Windows qua WinRM/PowerShell modules, nhưng cách xác thực, transport, privilege và module khác Linux SSH. Không lấy một playbook **apt**/**systemd** rồi chạy trên Windows. Khi học, tách **mô hình chung** (inventory, desired state, idempotency, batch, verify) khỏi **module theo OS**. Nếu một script PowerShell được Ansible gọi qua **win_shell**, bạn vẫn phải kiểm soát exit code, thay đổi thực, secret trong output và khả năng chạy lại. Với tác vụ phức tạp, ưu tiên module chuyên dụng có semantics trạng thái rõ thay vì shell script mơ hồ.

Câu hỏi: một playbook chạy trên 20 Linux VM và 5 Windows VM cần phân nhóm thế nào? Tách group/role/module theo OS, dùng biến kết nối đúng, test mỗi nhánh ở staging, giới hạn batch và kiểm tra dịch vụ sau thay đổi. “Ansible hỗ trợ cả hai” không có nghĩa cùng task chạy được cả hai.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.
- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.
- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://docs.ansible.com/projects/ansible/latest/getting_started/index.html
- https://github.com/kubernetes-sigs/kubespray/tree/master/docs